

10) find out the applications of recursion,

20) is recursion still used in the present programming era if so, where.

30) The Languages that support recursive codes?

1) Recursion is a programming technique in which a function calls itself.

2) Tree traverse: Recursion is commonly used to traverse tree data structures such as binary trees.

eg. Inorder traversal

Preorder traversal

Post order traversal

3) Searching and Sorting Algorithms: Many efficient algorithms use recursion.

eg. Binary Search

Merge Sort

Quick Sort

4) Mathematical computation: recursion definitions naturally model many mathematical problems.

eg. factorial calculation

Fibonacci Series

Greatest common divisor

5) Back tracking Problems: It helps to explore all solutions and back track when needed.

eg. Sudoku solver

Maze solving

5) Divide and Conquer Algorithms: Large problems are divided into smaller subproblems that are solved recursively.

eg. Merge sort

Quick sort

6) Graph Algorithms: used in graph traversal techniques

eg. Depth First Search

Topological Sorting

Finding connected components.

7) Dynamic programming: recursive solutions are optimized using memorization.

eg. Fibonacci with memorization

Longest common subsequence (LCS)

8) File System Operators: Directories often contain subdirectories, making recursion a natural choice.

9) Passing and compilers: nested language structures.

20) yes, recursion is still used in modern programming:

→ provides simple and elegant solution for complex problems

→ Tree traversal, graph traversal, divide-and-conquer algorithm

→ Quick sort, merge sort, Binary sort, Depth first sort.

→ Backtracking problems like Sudoku, N-Queens, and maze solving

→ Hierarchical data processing such as File System and HTML Structure.

Limitation:

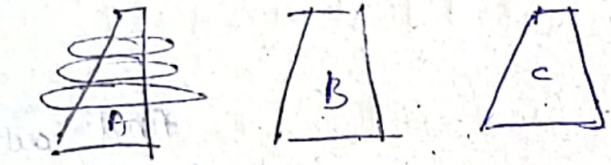
Recursion uses extra memory due to function calls and may cause stack overflow for very deep recursion.

(3)	C	JavaScript	Kotlin
	C++	PHP	Scala
	Java	Ruby	Haskell
	Python	Go	Rust
	C++	Swift	

Q) Point out the history of towers of Hanoi?

Q) Today, from the morning 7:30 AM to 4:30 PM list out various data structure in your surroundings.

Q1) towers of hanoi?



With help of B move all disks to C no small on big

```
#include <stdio.h>
```

```
void towerofHanoi (int n, char source, char destination, char Auxiliary){
```

```
if (n==1){
```

```
printf ("move disk 1 from %c to %c" source, destination);
```

```
return;
```

```
towerofHanoi (n-1, source, Auxiliary, destination);
```

```
printf ("move disk %d from %c to %c" n, source, destination);
```

```
towerofHanoi (n-1, Auxiliary, destination, source);
```

```
}
```

```
int main () {
```

```
int disk = 4;
```

```
printf ("sequence of disks to move from %c to %c", source, destination);
```

```
towerofHanoi (4, 'A', 'C', 'B');
```

```
return 0;
```


Q) Write a C++ code that includes a class student ^{find out} function.
the space occupied by member function / member variables.

Q). Find the frequency counts for the following

① For ($i=1$; $i < n$; $i++$);
Printf ("Hello");

② $i=1$;
 $i = i + 20$
Printf ("%d", i);

③ $i=1$;
while ($i \leq n$)
Printf ("%d", i);
 $i++$;
}

Q) for all the shell programs done in OS lab find out the time complexity in terms of frequency count.

Q1-A)

① for (i=1; i<n; i++);
printf("Hello");

for (i=1; i<n; i++)
 $\swarrow \quad \downarrow \quad \searrow$
 1 11 10 for n=10

for cost total
 $1 + n + 1 + n$ $2n + 2$

printf 1
 $\frac{2n + 3}{2}$

② i=1; // 1 unit
 i = i + 20; // 2 units
 printf("%d", i); // 1 unit

total = 4 units

③ i=1; // 1 time
 while (i<=n) { // n+1 units
 printf("%d", i); // n units
 i++; // n units
 }

total = $1 + n + 1 + n + n$
 $= 3n + 2$ units

Q-2-A)

①

```
# !/bin/bash
```

```
name="Siri" // 1 unit
```

```
echo "Hello, Siri!" // 1 unit
```

```
echo "welcome to os Subject" // 1 unit
```

$$T(n) = 1+1+1 = 3$$

②

```
# !/bin/bash
```

```
n=5 // 1 time
```

```
for ((i=1; i<=n; i++))
```

```
do
```

```
echo "current number: $i"
```

```
done
```

	Count	Repetition	Total
n=5	1	1	1
i=1	1	1	1
i<=n	1	n+1	n+1
i++	1	n	n
echo "current number: \$i"	1	n	n

$$3n+3$$

③

```
# !/bin/bash
```

```
n=3
```

```
for ((i=1; i<=n; i++))
```

```
do
```

```
for ((j=1; j<=n; j++))
```

```
do
```

```
echo "Coordinates: ($i, $j)"
```

```
done
```

```
done
```


	Cost	Repetition	total
$i=1$	1	2	1
$i \leq n$	1	$n+1$	$n+1$
$i++$	1	n	n
$j=1$	1	n	n
$j \leq n$	n	$n(n+1)$	n^2+n
$j++$	n	$n(n)$	n^2
echo	n	$n(n)$	n^2

$$1+n+1+2n+n^2+n+2n^2$$

$$= 3n^2+4n+2$$

Home work

Java generates hash code for every object. Find out what is the hash code in java and purpose of generating the hashCode.

→ Java inherited `hashCode()` method from the root `java.lang.Object` class.

Purpose $\begin{cases} \text{HashMap, HashSet, Hashtable} \\ \text{Ultra fast search operations} \\ \text{Quick Disambiguation before expensive checks} \end{cases}$

Equal objects must have equal hash codes.

`Obj1.equals(Obj2) → true`

`Obj2.hashCode()`

`Obj1.hashCode() → must equal to`

Unequal objects can share the same code.


```
Class Student{
```

```
    public class main{
```

```
        public static void main (String[] args) {
```

```
            Student s1 = new Student();
```

```
            System.out.println (s1.hashCode());
```

```
        }
```

```
    }
```

output:

1	5	1	6	4	6	7	6	8	8
---	---	---	---	---	---	---	---	---	---

Q) Trace Quick Sort as short as shown in the class for following array.

> 85, 76, 99, 13, 45, 103, -5, 26, 31

> 350, 790, 400, 200, 100, 96, 900, 890, 760

85	76	99	13	45	103	-5	26	31
----	----	----	----	----	-----	----	----	----

i j i j

$i=99$ $j=31$ swap

85	76	31	13	45	103	-5	26	99
----	----	----	----	----	-----	----	----	----

i j i j i j

$i=103$ $j=26$ swap

85	76	31	13	45	26	-5	103	99
----	----	----	----	----	----	----	-----	----

i j

$i > j$

swap $arr[pivot]$ $arr[j]$

-5	76	31	13	45	26	85	103	99
----	----	----	----	----	----	----	-----	----

~~do it like this~~

i j

no change

-5	76	31	13	45	26	85	103	99
----	----	----	----	----	----	----	-----	----

Right sub segment

pivot = 76

-5	76	31	13	45	26	85	103	99
----	----	----	----	----	----	----	-----	----

i j i j i j i j

$i > j$

-5	26	31	13	45	76	85	103	99
----	----	----	----	----	----	----	-----	----

i j i j i j i j

-5	26	13	31	45	76	85	103	99
----	----	----	----	----	----	----	-----	----

j i j j j j j

j j

$i > j$

-5	13	26	31	45	76	85	103	99
----	----	----	----	----	----	----	-----	----

j i j j j j j j

j j

$i > j$

same upto 85

-5	13	26	31	45	76	85	103	99
----	----	----	----	----	----	----	-----	----

j i
 j j

-5	13	26	31	45	76	85	99	103
----	----	----	----	----	----	----	----	-----

②

305	790	400	200	100	96	900	890	760
-----	-----	-----	-----	-----	----	-----	-----	-----

j i j j j j j j

305	96	400	200	100	790	900	890	760
-----	----	-----	-----	-----	-----	-----	-----	-----

j j i j j j j j

305	96	100	200	400	790	900	890	760
-----	----	-----	-----	-----	-----	-----	-----	-----

j j j j i j j j j

200	96	100	305	400	790	900	890	760
-----	----	-----	-----	-----	-----	-----	-----	-----

j j j i j j j j j

100	96	200	305	400	790	900	890	760
-----	----	-----	-----	-----	-----	-----	-----	-----

j j i j j j j j j

96	100	200	305	400	790	900	890	760
----	-----	-----	-----	-----	-----	-----	-----	-----

j i j

96	100	200	305	400	790	760	890	900
----	-----	-----	-----	-----	-----	-----	-----	-----

j j i j

Quick Sort tracing for same question

54	12	19	3	8	16	31
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
						j

31	12	19	3	8	46	54
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
				j		

8	12	19	3	31	46	54
$\uparrow$	$\uparrow$		$\uparrow$			
			j			

8	3	19	12	31	46	54
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$			
		j				

3	8	19	12	31	46	54
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$

3	8	12	19	31	46	54
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$

Insertion sort - 2 1 3 21 51

#include <stdio.h>

void insertionSort(int arr[], int n)

{ for (int i=1; i<n; i++) {

int key = arr[i];

int j = i-1;

while (j>0 && key < arr[j]) {

arr[j+1] = arr[j];

j--;

}

arr[j+1] = key;

}

}